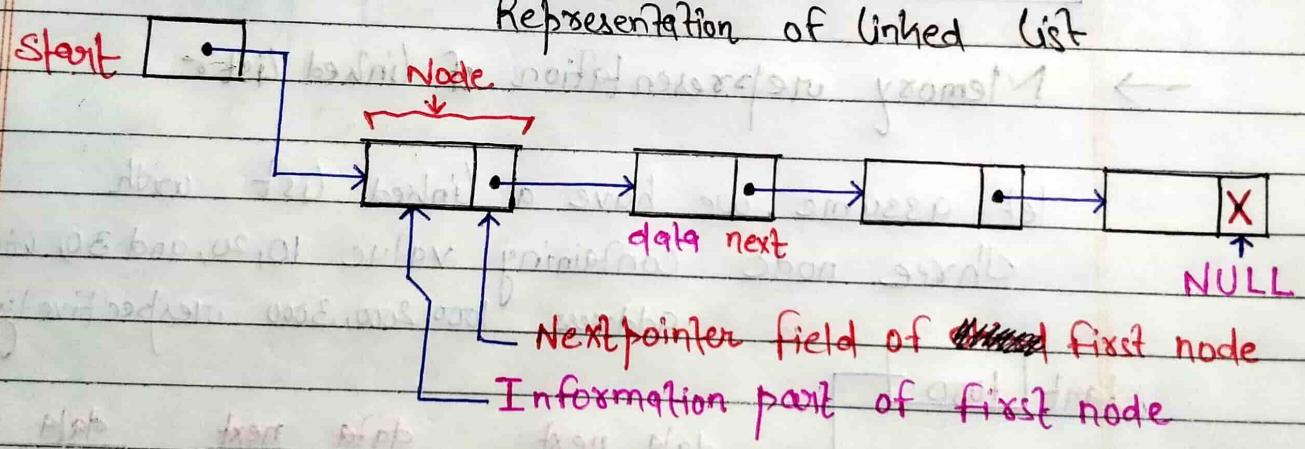
Linked - List

→ A Linked List is a linear data structure where elements (nodes) are connected using pointers. Each node contains two parts:

1.) Data - The actual value stored in the node.
2.) Pointer - A reference to the next node in the sequence.

Unlike array, linked list do not require contiguous memory allocation and can grow or shrink dynamically.

Representation of linked list



- In actual, 0 or a negative number is used for the null pointer.
  The null pointer, denoted by X in the diagram, signals the end of the list.

- The first part is the information part of the node, which can store any type of information, such as integer, character, or objects.

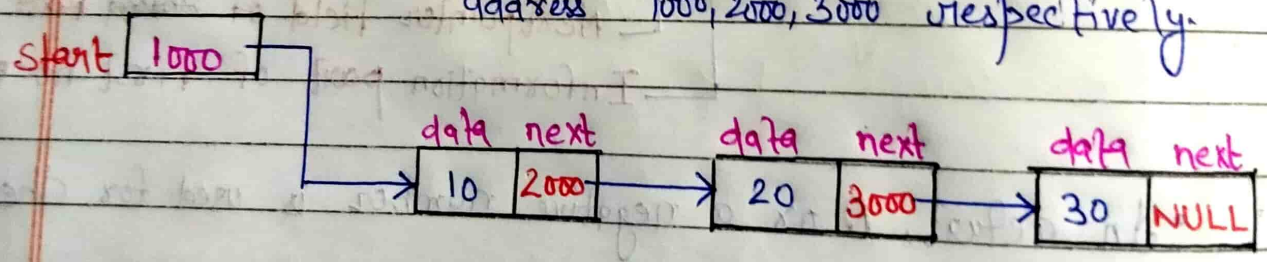
Akash

- The second part called next pointer field, containing the address of the next node of the list.
- The linked list also contains a list pointer variable called start/head which contains the address of ~~this~~ the first node in a list.

> Note:~
> If a list ~~thas~~ that has no nodes, such a list is called null list or empty list and is denoted by the null pointer in the variable start/head.

→ Memory representation of linked list:-

let assume, we have a linked list with three node containing value 10, 20, and 30, with address 1000, 2000, 3000 respectively.



Address	Data	Next (pointer to next node)
1000	10	2000
2000	20	3000
3000	30	NULL

Akash

→ Advantages of Linked List :-

- Dynamic size: Can grow or shrink as needed.
- Efficient Insertions/Deletions: No need to shift element like in arrays.
- Memory Utilization: No pre-allocation required.

→ Disadvantages of Linked list :-

- More memory usage: Requires extra storage for pointer.
- Slower Access time: Cannot access elements directly like arrays (must traverse the list).

→ Types of Linked Lists :-

1.) Singly Linked List - Each node has a data field and a pointer to the next node.

2.) Doubly Linked List - Each node has a data field and a pointer to both the next and previous nodes.

3.) Circular Linked List - The last node's next pointer points to the first node.

→ Key operation in Linked List :

1.) Insertion	2.) Deletion	3.) Traversal
• At the begining	• from the begining	• printing all elements
• At the end	• From the end	• searching for an element
• At a specific position	• By a specific position.	

Akash